

Practical 03

Choosing a Model Honestly

Overfitting, the train/test gap,
and cross-validation

Student	Lakhan Gupta
Enrolment number	S24CSEU0067
Programme	B.Tech CSE · 5th Semester
Institution	Bennett University · Session 2026-27
Course outcome	CO2 · follows lectures L04-L05
Notebook	P03_S24CSEU0067.ipynb
Self-check result	8 of 8 checks passed

SOURCE CODE REPOSITORY

https://github.com/LakhanxGupta/MLOPS_BU

https://github.com/LakhanxGupta/MLOPS_BU/tree/main/P03_lab

01 Step 0 - Check the workbench

Preflight: Python 3.13.14 with numpy, pandas, scikit-learn and matplotlib all importable.

```
In [1]: # Step 0 --- check the workbench before we start.  
# This cell only looks; it changes nothing. Run it and read the last line.
```

```
import sys  
from pathlib import Path  
  
print("Python  :", sys.version.split()[0])  
print("Folder  :", Path.cwd().name)  
  
_missing = []  
for _name in ['numpy', 'pandas', 'sklearn', 'matplotlib']:  
    try:  
        __import__(_name)  
    except ImportError:  
        _missing.append(_name)  
  
for _name in ['numpy', 'pandas', 'sklearn', 'matplotlib']:  
    _mark = "missing" if _name in _missing else "ok"  
    print(f"  {_name:<14} {_mark}")  
  
if _missing:  
    print()  
    print("STOP. Some libraries are missing:", ", ".join(_missing))  
    print("Ask your instructor to run the setup in labs/SETUP.md.")  
else:  
    print()  
    print("All good. You can carry on to Step 1.")
```

```
Python  : 3.13.14  
Folder  : P03-model-choice  
numpy    ok  
pandas   ok  
sklearn  ok  
matplotlib ok
```

All good. You can carry on to Step 1.

02 Step 0b - The dataset

Confirms the shared 600-row delivery dataset is present; rebuilds it from seed 42 only if missing.

```
In [2]: # The delivery-time dataset every practical in this course uses.
# If the file is missing we build it again from the same seed, so every
# student in the room gets byte-for-byte the same 600 rows.

import csv
from pathlib import Path

import numpy as np

SEED = 42
N_ROWS = 600
DATA = Path("../") / "data" / "delivery_times.csv"

def make_delivery_csv(path=DATA):
    """Write the 600-row delivery dataset. Same formula as the lectures."""
    rng = np.random.default_rng(SEED)
    distance_km = np.round(rng.uniform(0.5, 12.0, N_ROWS), 2)
    prep_time_min = np.round(rng.uniform(5, 30, N_ROWS), 0)
    traffic_level = rng.integers(1, 4, N_ROWS)
    rain = rng.binomial(1, 0.25, N_ROWS)
    delivery_min = np.round(
        6.0
        + 3.1 * distance_km
        + 0.65 * prep_time_min
        + 4.2 * traffic_level
        + 5.5 * rain
        + rng.normal(0, 2.5, N_ROWS),
        1,
    )
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["distance_km", "prep_time_min", "traffic_level",
                    "rain", "delivery_min"])
        for i in range(N_ROWS):
            w.writerow([distance_km[i], int(prep_time_min[i]),
                        int(traffic_level[i]), int(rain[i]), delivery_min[i]])
    return path

if not DATA.exists():
    make_delivery_csv()
    print("dataset rebuilt ->", DATA)
else:
    print("dataset found ->", DATA)
```

dataset found -> ../data/delivery_times.csv

03 Step 1 - Set up the data once, for all models

Same features, target and 80/20 split with random_state=42 as P02: 480 training and 120 test orders.

```
In [3]: import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

orders = pd.read_csv(DATA)

FEATURES = ["distance_km", "prep_time_min", "traffic_level", "rain"]
X = orders[FEATURES]
y = orders["delivery_min"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("train:", len(X_train), " test:", len(X_test))
```

train: 480 test: 120

04 Step 2 - A helper that scores a model both ways

`score_both_ways()` fits a model, then reports its train MAE, test MAE and the gap between them.

```
In [4]: from sklearn.metrics import mean_absolute_error

def score_both_ways(model, name):
    """Train the model, then score it on train and on test."""
    model.fit(X_train, y_train)
    train_mae = mean_absolute_error(y_train, model.predict(X_train))
    test_mae = mean_absolute_error(y_test, model.predict(X_test))
    gap = test_mae - train_mae
    print(f'{name:<26} train {train_mae:5.2f}  '
          f'test {test_mae:5.2f}  gap {gap:5.2f}')
    return {"name": name, "train": train_mae,
           "test": test_mae, "gap": gap}

print("helper ready")
helper ready
```

05 Step 3 - Model 1: linear regression, the honest baseline

Train 2.04, test 1.92, gap -0.11: a straight line is too simple to memorise, so it does not overfit.

```
In [5]: from sklearn.linear_model import LinearRegression

linear = score_both_ways(LinearRegression(), "LinearRegression")

LinearRegression      train  2.04   test  1.92   gap -0.11
```

06 Step 4 - Model 2: a decision tree with no limits

Train 0.00, test 3.43, gap 3.43: the unlimited tree memorised every training order. That gap is overfitting.

```
In [6]: from sklearn.tree import DecisionTreeRegressor

wild_tree = score_both_ways(
    DecisionTreeRegressor(random_state=42), "DecisionTree (no limit)"
)

DecisionTree (no limit)  train  0.00   test  3.43   gap  3.43
```

07 Step 5 - Model 3: the same tree, but shallow

`max_depth=4`: train 3.66, test 4.23, gap 0.57. Limiting the tree shrank the gap sharply.

```
In [7]: small_tree = score_both_ways(
    DecisionTreeRegressor(max_depth=4, random_state=42),
    "DecisionTree (depth 4)",
)

DecisionTree (depth 4)  train  3.66   test  4.23   gap  0.57
```

08 Step 6 - Model 4: a random forest

50 averaged trees: train 0.99, test 2.40, gap 1.40, far healthier than a single unlimited tree.

```
In [8]: from sklearn.ensemble import RandomForestRegressor

forest = score_both_ways(
    RandomForestRegressor(n_estimators=50, random_state=42),
    "RandomForest (50 trees)",
)

RandomForest (50 trees)  train  0.99   test  2.40   gap  1.40
```

09 Step 7 - Put the four results side by side

Sorted by test MAE, LinearRegression is best at 1.92 minutes; the gap column is read alongside it.

```
In [9]: results = pd.DataFrame([linear, wild_tree, small_tree, forest])
results = results.round(2).sort_values("test")

print(results.to_string(index=False))
print()
best = results.iloc[0]
print(f"Lowest test MAE: {best['name']} at {best['test']:.2f} minutes")
```

	name	train	test	gap
	LinearRegression	2.04	1.92	-0.11
	RandomForest (50 trees)	0.99	2.40	1.40
	DecisionTree (no limit)	0.00	3.43	3.43
	DecisionTree (depth 4)	3.66	4.23	0.57

Lowest test MAE: LinearRegression at 1.92 minutes

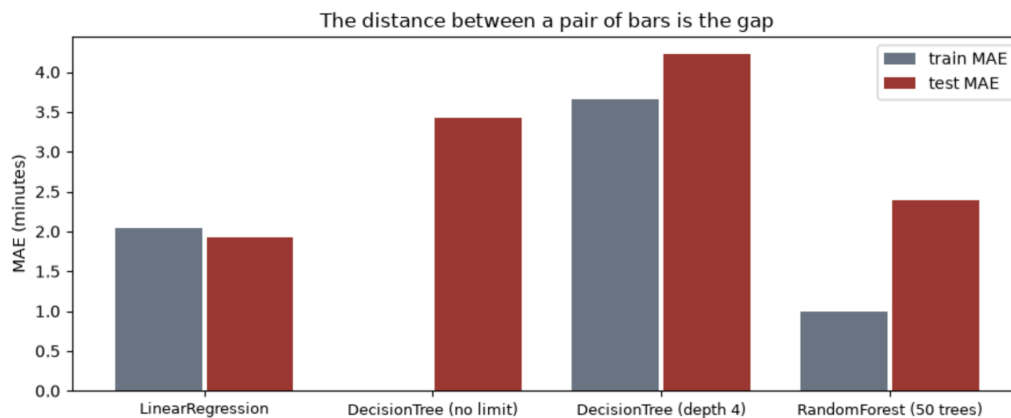
10 Step 8 - Draw the gap

Grouped train/test bars: the unlimited tree's train bar vanishes while its test bar stays tall.

```
In [10]: %matplotlib inline
import matplotlib.pyplot as plt

four = [linear, wild_tree, small_tree, forest]
xs = np.arange(len(four))

fig, ax = plt.subplots(figsize=(9, 3.8))
ax.bar(xs - 0.2, [m["train"] for m in four], 0.38,
       label="train MAE", color="#6B7785")
ax.bar(xs + 0.2, [m["test"] for m in four], 0.38,
       label="test MAE", color="#9E3B33")
ax.set_xticks(xs, [m["name"] for m in four], fontsize=9)
ax.set_ylabel("MAE (minutes)")
ax.set_title("The distance between a pair of bars is the gap")
ax.legend()
plt.tight_layout()
plt.show()
```



11 Step 9 - Stop trusting one split: cross-validation

5-fold MAE: LinearRegression 2.03 (+/- 0.16), RandomForest 2.69 (+/- 0.40), DecisionTree(4) 4.57 (+/- 0.18).

```
In [11]: from sklearn.model_selection import cross_val_score

def cross_validate(model, name):
    """Average MAE over 5 different train/test splits."""
    scores = -cross_val_score(
        model, X, y, cv=5, scoring="neg_mean_absolute_error"
    )
    print(f"{name:<26} MAE {scores.mean():5.2f} "
          f"+/- {scores.std():4.2f}")
    return scores.mean()

cv_linear = cross_validate(LinearRegression(), "LinearRegression")
cv_tree = cross_validate(
    DecisionTreeRegressor(max_depth=4, random_state=42),
    "DecisionTree (depth 4)"
)
cv_forest = cross_validate(
    RandomForestRegressor(n_estimators=50, random_state=42),
    "RandomForest (50 trees)"
)

LinearRegression      MAE  2.03 (+/- 0.16)
DecisionTree (depth 4) MAE  4.57 (+/- 0.18)
RandomForest (50 trees) MAE  2.69 (+/- 0.40)
```

12 Step 10 - Tune one setting, properly

Tree depth tuned by cross-validation, never on the test set: best depth is 8 at MAE 3.39; depth 2 underfits at 5.86.

```
In [12]: depths = [2, 3, 4, 6, 8, None]
scores_by_depth = {}

for depth in depths:
    tree = DecisionTreeRegressor(max_depth=depth, random_state=42)
    mae = -cross_val_score(
        tree, X, y, cv=5, scoring="neg_mean_absolute_error"
    ).mean()
    scores_by_depth[depth] = round(mae, 3)
    print(f"max_depth={str(depth):<5} MAE {mae:5.2f}")

best_depth = min(scores_by_depth, key=scores_by_depth.get)
print()
print("best depth:", best_depth)

max_depth=2      MAE  5.86
max_depth=3      MAE  4.85
max_depth=4      MAE  4.57
max_depth=6      MAE  3.64
max_depth=8      MAE  3.39
max_depth=None   MAE  3.47

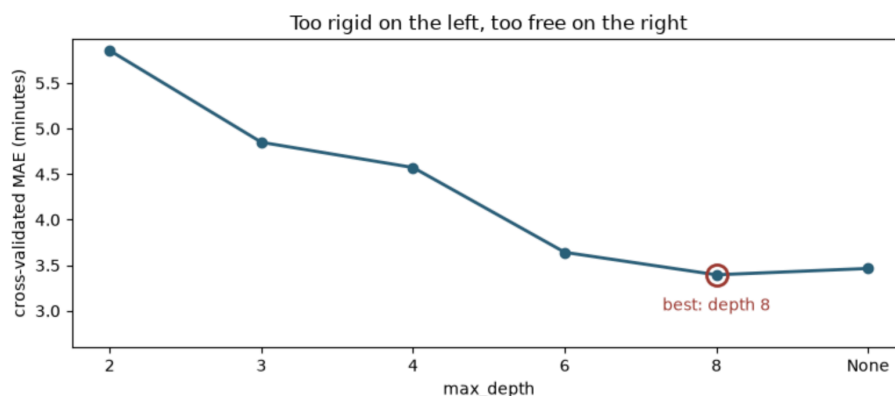
best depth: 8
```

13 Step 11 - Draw the sweep

The U-shaped curve: underfitting on the left, a minimum at depth 8, and a slight rise again with no limit.

```
In [13]: xs = np.arange(len(depths))
maes = [scores_by_depth[d] for d in depths]
best_i = int(np.argmin(maes))

fig, ax = plt.subplots(figsize=(8, 3.6))
ax.plot(xs, maes, marker="o", color="#2C5F79", linewidth=2)
ax.plot(xs[best_i], maes[best_i], marker="o", markersize=13,
        markerfacecolor="none", markeredgewidth=2,
        markeredgewidth=2)
ax.annotate(f"best: depth {depths[best_i]}",
            (xs[best_i], maes[best_i]),
            xytext=(0, -22), textcoords="offset points",
            ha="center", color="#9E3B33")
ax.set_xticks(xs, [str(d) for d in depths])
ax.set_ylim(bottom=min(maes) - 0.8)
ax.set_xlabel("max_depth")
ax.set_ylabel("cross-validated MAE (minutes)")
ax.set_title("Too rigid on the left, too free on the right")
plt.tight_layout()
plt.show()
```



14 Step 12 - Choose, and be able to say why

LinearRegression chosen on cross-validated error, training cost and explainability.

```
In [14]: print("Cross-validated MAE, best first:")
for name, mae in sorted(
    {"LinearRegression": cv_linear,
     "DecisionTree(4)": cv_tree,
     "RandomForest(50)": cv_forest}.items(),
    key=lambda kv: kv[1],
):
    print(f" {name:<20} {mae:5.2f}")

print()
print("Our choice for the delivery service: LinearRegression.")
print("Reasons:")
print(" 1. It has the lowest cross-validated error here.")
print(" 2. It trains in milliseconds and needs almost no memory.")
print(" 3. We can read its coefficients and explain a prediction")
print("    to a restaurant owner. Nobody can read 50 trees.")
print()
print("Be careful with reason 1. This dataset really was built")
print("from a straight line, so a straight line wins. On messier")
print("real data the forest usually wins instead -- and then you")
print("must decide what its extra accuracy is worth, against")
print("reasons 2 and 3, which do not change.")
```

```
Cross-validated MAE, best first:
LinearRegression      2.03
RandomForest(50)     2.69
DecisionTree(4)       4.57
```

```
Our choice for the delivery service: LinearRegression.
Reasons:
```

1. It has the lowest cross-validated error here.
2. It trains in milliseconds and needs almost no memory.
3. We can read its coefficients and explain a prediction to a restaurant owner. Nobody can read 50 trees.

Be careful with reason 1. This dataset really was built from a straight line, so a straight line wins. On messier real data the forest usually wins instead -- and then you must decide what its extra accuracy is worth, against reasons 2 and 3, which do not change.

15 Task T1 - Measure overfitting yourself

A depth-12 tree scores train 0.05 and test 3.40, a train/test gap of 3.35 minutes.

```
In [15]: deep_tree = DecisionTreeRegressor(max_depth=12, random_state=42)

# Fit the tree on the training data
deep_tree.fit(X_train, y_train)

# Measure the MAE on the training data
T1_train_mae = mean_absolute_error(y_train, deep_tree.predict(X_train))

# Measure the MAE on the test data
T1_test_mae = mean_absolute_error(y_test, deep_tree.predict(X_test))

# The gap between them
T1_gap = T1_test_mae - T1_train_mae

print(f"train {T1_train_mae:.2f} test {T1_test_mae:.2f} gap {T1_gap:.2f}")
train 0.05 test 3.40 gap 3.35
```

16 Task T2 - Tune the forest, without touching the test set

5-fold CV over max_depth [2, 4, 6, None]: MAE 5.27, 3.64, 2.84, 2.69, so the best setting is None.

```
In [16]: T2_depths = [2, 4, 6, None]

# Cross-validate a 50-tree forest at each depth
T2_scores = {}
for depth in T2_depths:
    forest_cv = RandomForestRegressor(
        n_estimators=50, max_depth=depth, random_state=42
    )
    mae = -cross_val_score(
        forest_cv, X, y, cv=5, scoring="neg_mean_absolute_error"
    ).mean()
    T2_scores[depth] = round(mae, 3)

# Which depth had the lowest MAE?
T2_best_depth = min(T2_scores, key=T2_scores.get)

print(T2_scores)
print("best:", T2_best_depth)

{2: np.float64(5.271), 4: np.float64(3.643), 6: np.float64(2.844), None: np.float64(2.689)}
best: None
```

17 Task T3 - A comparison you can reuse

compare() takes {name: model} and returns test MAE: linear 1.92, tree4 4.23, forest 2.40.

```
In [17]: def compare(models):
    """Fit each model, score it on the test set, return {name: mae}."""
    results = {}
    for name, m in models.items():
        m.fit(X_train, y_train)
        results[name] = mean_absolute_error(y_test, m.predict(X_test))
    return results

T3_table = compare({
    "linear": LinearRegression(),
    "tree4": DecisionTreeRegressor(max_depth=4, random_state=42),
    "forest": RandomForestRegressor(n_estimators=50, random_state=42),
})

print(T3_table)

{'linear': 1.9246530798338346, 'tree4': 4.228332168713071, 'forest': 2.398883333333333}
```

18 Self-check - marking table

Self-check: 8 of 8 checks passed.

```
In [18]: # -----
# SELF-CHECK --- run this when you have attempted the tasks above.
# It never breaks your notebook. A task you have not done yet simply
# shows FAIL.
# -----

_results = []

def _check(label, fn):
    """Evaluate one graded condition without ever raising."""
    try:
        ok = bool(fn())
    except Exception:
        ok = False
    _results.append((label, ok))

_check('T1 | the depth-12 tree is trained', lambda: hasattr(deep_tree, 'tree_'))
_check('T1 | T1_train_mae and T1_test_mae are that tree's two scores', lambda: abs(float(T1_train_mae) - mean_absolute_error(y_train,
deep_tree.predict(X_train))) < 0.01 and abs(float(T1_test_mae) - mean_absolute_error(y_test, deep_tree.predict(X_test))) < 0.01)
_check('T1 | T1_gap is test minus train, and it is positive', lambda: abs(float(T1_gap) - (float(T1_test_mae) - float(T1_train_mae))) <
1e-6 and float(T1_gap) > 0)
_check('T2 | T2_scores has one score per depth', lambda: set(T2_scores) == {2, 4, 6, None})
_check('T2 | every score is a plausible MAE in minutes', lambda: all(0 < float(v) < 20 for v in T2_scores.values()))
_check('T2 | T2_best_depth is the depth with the lowest score', lambda: T2_best_depth == min(T2_scores, key=T2_scores.get))
_check('T3 | compare() returns one score per model', lambda: set(T3_table) == {'linear', 'tree4', 'forest'})
_check('T3 | the linear score matches the walkthrough', lambda: abs(float(T3_table['linear']) - linear['test']) < 0.01)

print("=====")
print("SELF-CHECK   Practical 03 --- Choosing a Model Honestly")
print("=====")
for _label, _ok in _results:
    print(f"  [{'PASS' if _ok else 'FAIL'}]  {_label}")
print("=====")
_passed = sum(1 for _, _ok in _results if _ok)
print(f"  {_passed} of {len(_results)} checks passed")
print("=====")
if _passed == len(_results):
    print("Well done. Save the notebook and submit it.")
else:
    print("Read the FAIL lines above, fix those tasks, run this cell again.")

=====
SELF-CHECK   Practical 03 --- Choosing a Model Honestly
=====
[PASS] T1 | the depth-12 tree is trained
[PASS] T1 | T1_train_mae and T1_test_mae are that tree's two scores
[PASS] T1 | T1_gap is test minus train, and it is positive
[PASS] T2 | T2_scores has one score per depth
[PASS] T2 | every score is a plausible MAE in minutes
[PASS] T2 | T2_best_depth is the depth with the lowest score
[PASS] T3 | compare() returns one score per model
[PASS] T3 | the linear score matches the walkthrough
=====

  8 of 8 checks passed
=====
Well done. Save the notebook and submit it.
```

19 Conclusion - Which model I would ship

The three-sentence answer required under What to submit.

Which model I would ship

I would put **LinearRegression** in front of real customers. It had the lowest 5-fold cross-validated MAE (2.03 ± 0.16 minutes, against 2.69 ± 0.40 for the 50-tree random forest and 4.57 for the depth-4 tree), it showed essentially no train/test gap (2.04 train vs 1.92 test), and its four coefficients let us explain any single prediction to a restaurant owner. By choosing it I gave up the ability to learn non-linear effects and interactions (for example, rain mattering more in heavy traffic), which a random forest could capture if real delivery data turns out to be messier than this dataset.